

Classification-based Adaptive Web Scraper

Ujwal B V S^{*†}, Bharat Gai^{*†}, Abhishek Kundu[†], Anusha Holla[†] and Mukund Rungta[‡]

^{*}Both have contributed equally

[†]Samsung Research Institute, Bangalore, India

Email: {ujwal2006, bharatgai234, kundu.abhishek, anusha.holla}@gmail.com

[‡]Indian Institute of Technology, Delhi, New Delhi, India

Email: mukundrungta000@gmail.com

Abstract—Web scraping is an important problem in computer science. The problem with the commonly-used position or structure-based web scraping tools is that they need to be manually reconfigured as soon as the structure of the web page changes. In this paper, we try to solve this problem of information extraction for web pages consisting of repetitive blocks. We extract these blocks and their constituent attributes, using a novel classification-based approach. Our approach gives high accuracy when used to extract product-offers from an offer-aggregator website. It is also highly adaptive to the changing structure of a website.

I. INTRODUCTION

Web scraping is an ever-growing, important problem in computer science, with a plethora of applications in both business and research domains. In many such applications, it is very important to extract lists and its constituent attributes from a web page. Building a product-offer-based recommendation system, which scrapes its content from a variety of websites (containing lists of offers) and recommends the best deals to users, is one such example. Secondly, web pages (and their HTML structures) are susceptible to change, simply because the look-and-feel of the website needs to evolve with time. Due to this, the commonly-used scraping tools need to be manually reconfigured every time, since they are position or structure-based (and use CSS Selectors or xpaths).

In this paper, we try to build an automatic web-scraping system, that is adaptive to structure change. Our approach uses domain knowledge (by training a domain-specific classifier) as an input to our algorithm, which is independent of the same. We solve the problem of information extraction, for web pages consisting of repetitive blocks, with each block representing an object (a product-offer, in our case) and containing its attributes (offer-title, offer-description, offer-expiry, etc.); using a novel classification-based approach. Our problem statement can be divided into two parts: extracting these repetitive blocks and retrieving their internal attributes. For this purpose, we classify the text of the DOM tree of the web page, into one of these attributes/labels (or *OTHER*, for text that is unrelated to the object) and look for repetitive tree-structures or blocks in the web page, that contain all the attributes of a single object. We, then find all the remaining occurrences of these tree-structures (to account for the misclassification errors) in the original DOM tree, and subsequently, get all the required objects/blocks (with their attributes) from this web page. If the structure of the web page,

and therefore the structure of the repetitive blocks change in the future, new corresponding repetitive tree-structures are extracted automatically, which makes our approach adaptive to structure-change. This happens because our approach is content-based and not position/location-based.

We tested our approach by applying it to the specific problem of extracting product-offers from various web pages of an offer-aggregator website and achieved remarkable results. Then, we tested it on older versions of these web pages, having completely different structures and still got a high accuracy, which proves our claims experimentally as well. Some other useful applications of our approach could be extracting product-information from e-commerce websites; flight-information from travel company websites; or real-estate listings.

II. RELATED WORK

The field of web scraping has been worked upon substantially. A lot of research has been done on wrapper-generation, from a long time. A wrapper is a program which extracts relevant information from a web page. Various wrapper induction [1] methods like WIEN [2], Softmealy [3], Stalker [4] have been proposed. Many of these methods are based on a supervised learning approach, which uses labeled web pages to learn the rules (that are structure or position-based) for data extraction, so the web pages need to be manually labeled every time their structure changes. Our approach also uses supervised learning but only for classifying text. This content-based learning decouples our method from the HTML structure, making it immune to structural change.

[5] describes a method to completely automate the wrapper generation process by using Linked Data and achieved an accuracy of over 80%.

[6] defines DEPTA, a two-step approach to first extract the data regions (a set of data records) using visual information, and then getting the data records from the regions using partial tree alignment. Our approach uses domain knowledge (a domain-specific classifier) to extract both the data records (rows or objects) as well as the exact data items (columns or the object's attributes), thus completely automating the process (of finding labeled attributes).

[7] tries to first identify a subject by extracting meta data present in the DOM tree of the web page, like *h1*, *h2* or *title* tags and then computes the relevance of data *u*:

node density. [8] is another domain-independent approach. The author uses a method, Data Path Matching to identify similar sub-trees in the DOM hierarchy and clusters them (into a set of data records) and then removes unwanted data using visual information. Data Path Code Alignment is then used to extract data items.

[9] proposes a method to extract relevant detailed information from a website, given multiple pages. It is useful for Information Extraction from web sites like Amazon for getting product details. [10] further annotates the extracted information using heuristics. However, the problem with this approach is its dependence on multiple similar web pages, whereas our approach just needs a single web page to extract all its data records along with their attributes.

[11] gives a domain independent approach for extracting table like structures. The author works under the strong assumption that each data record contains a link to a detailed page. Also, multiple similar pages are needed to get the slots containing the table. Two different approaches (CSP and Probabilistic Inference) for getting the data records are explained.

III. DESIGN AND ALGORITHM

Before we proceed with the details of our approach, it is important to formally define some of the terms, we'll use throughout the paper. A **repetitive block** or a **block** is any HTML element in the DOM hierarchy, which repeats in the web page, along with its descendants (some descendants, if not all). Such an element can be mapped to a real **object** (a product-offer, for example) and the information (text or image) within its descendants can be mapped to that object's **attributes** (offer-title, offer-description, offer-benefits etc.). A collection of such repetitive blocks, representing different instances of a particular object, forms a **list**. Note that there could be multiple lists in a web page representing different objects, but our algorithm will be dealing with extracting only one list at a time. An **attribute-container** refers to the HTML tag in the DOM tree representing an attribute. A *div* containing an offer's description in its subtree is an example of the same. A **label** l_i corresponds to an attribute to be extracted and is predefined by the user. For example, *TITLE* is a label that corresponds to an offer's title (an attribute), whereas the *OTHER* label is given to unrelated text. We use **path** to locate a node in a tree structure. It could either be an xpath or a CSS-selector. Also, we define the path we use to locate an attribute as an **attribute-path** and is the final output of our algorithm. We will extensively use the example of the product-offer object, its attributes (offer-title, offer-description, offer-benefits, offer-expiry, offer-coupon-code, offer-tags) and their corresponding labels (*TITLE*, *DESC*, *BENEFITS*, *EXPIRY*, *COUPON_CODE*, *TAGS*) throughout this paper, to simplify understanding. However, the proposed algorithm is generic and can be used to extract any list of objects of the same type, contained in a webpage.

The first step in our approach, that is essential and used as an input to the proposed algorithm is developing a website-

specific (or domain-specific) classifier, that labels any given piece of text taken from the web page into one of the predefined labels. This classifier is trained only once. Every time the structure of the page is changed, the algorithm is rerun using the same classifier to find the new repetitive blocks and their corresponding new attribute-containers. Such changes can be detected automatically by periodically using the classifier to validate the attributes being extracted; and by checking if the attribute-paths stored (when the algorithm was last run), still exist or not.

A. Training a classifier

It involves the following steps :

1) *Data collection*: To develop an efficient classifier, we need a relevant and labeled training data set. To extract this data set, we have written a css-selector-based script, using the python tool Beautiful Soup. It traverses through all the nodes in the DOM tree associated with a particular attribute and extracts its text (In case, it is not a leaf node, its text is calculated by recursively appending the text of all its descendants in a left to right fashion). The text of all the leaf nodes that are not associated with any of the attributes is extracted under the label *OTHER*. 80% of this data set generated is used to train the classifier (with the other 20% being used for testing).

Note that the extraction done using Beautiful Soup is just a way to train the classifier once. The developer is free to train the classifier in a different way and feed it as an input to our proposed algorithm (explained in the next section), which is completely website-independent. Consider a case when there is already a large data set of objects (and their attributes) present in a database. The developer can directly use this data to train the classifier and thus, in this case, there is no need to scrape a webpage using a path-based script.

2) *Data Cleaning and Preprocessing*: The extracted data needs to be cleaned, before being further used for training. This process is highly domain-specific. For example, in case of an offers-website, we detect all occurrences of coupon codes using regex (any alphanumeric capitalized string of length greater than or equal to 4, having at least two alphabets) and replace each with the string “\$COUPON_CODE\$”; all occurrences of dates and replace each with “\$DATE\$”; and all occurrences of numbers and replace each with “\$NUMBER\$”. Finally, we remove all the stopwords from the data set.

3) *Conversion to Feature Vectors*: The cleaned data now needs to be converted into feature vectors, that the classifier actually understands. Using CountVectorizer of scikit-learn library in python, we have created a vocabulary with a maximum of F_{max} features (most commonly occurring words) by iterating on the output of the previous step. Each of the entry (text) in the data set is then converted into an array of maximum F_{max} features. Finally, the length of the original text (before removing stopwords), is added as one of the features. In the merchant offers example, the value of F_{max} is 5000.

4) *Training a Random Forest Classifier*: A Random Forest classifier with 50 estimators is trained, using the feature

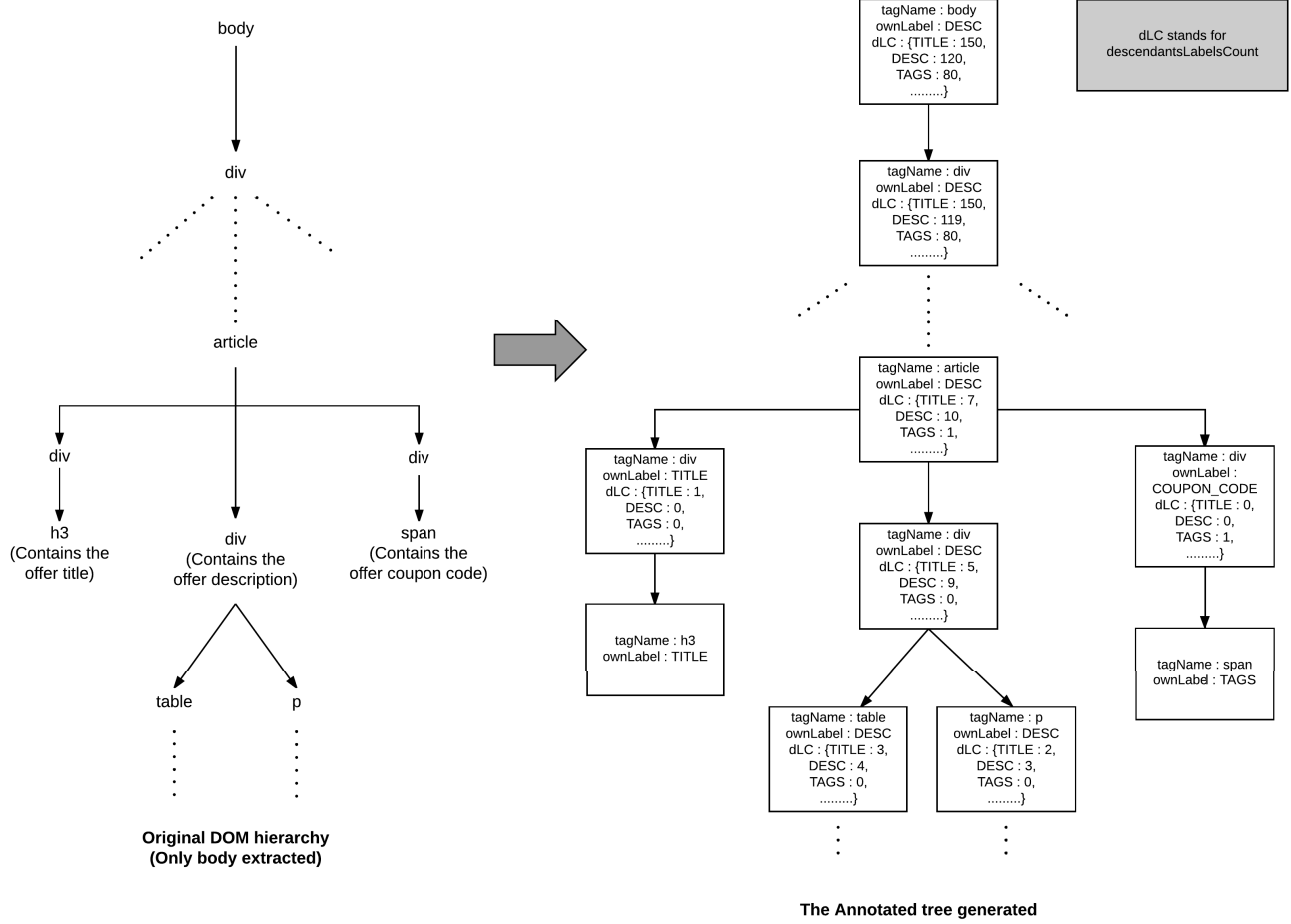


Fig. 1. Constructing an Annotated Tree

vectors formed above.

Various heuristics can be used to improve the accuracy of the classifier, that is domain/website-specific. In the offers example, the labels *COUPON_CODE* and *EXPIRY* can be directly detected using regular expressions, if the length of the input text is less than equal to 4 and it contains the string “\$*COUPON_CODE*\$” or “\$*DATE*\$”. “Ends on 24 Jan 2018” can directly be labeled as *EXPIRY* and “Coupon Code: CASHBACK20” as *COUPON_CODE*.

B. The Proposed Algorithm

Our algorithm takes as input three parameters: a list of compulsory labels, a list of optional labels and a website-specific classifier (like the one trained in the previous step). A **compulsory label** (l_c) represents an attribute, the text of whose attribute-container is never an empty string and is always relevant to the attribute. For example, the label *TITLE* must be specified as a compulsory label to the algorithm, because every offer has a title and the attribute-container (some *div*, for instance) that stores this offer title will never be empty. Any label that is not a compulsory label is an **optional label** (l_o).

The attribute offer-coupon-code, for instance, might be empty in some offers (the offers that don’t need a coupon code). So, *COUPON_CODE* is an optional label. Another example is the attribute offer-expiry, whose attribute-container might contain the string, “Not Applicable” in some of the blocks. Now, this string is irrelevant to offer-expiry, because it does not contain a date format. So, *EXPIRY* is an optional label.

One important point that needs to be clarified here is that only the attributes having the same **internal path** (path from the block’s root tag till the attribute-container) of their respective attribute-containers in each of the repetitive blocks of a list, can be chosen. In other words, the user must not choose attributes, whose attribute-containers lie in the non-repetitive structure (which is not present in the rest of the members of the list) of a block. A simple rationale behind this requirement is that when a dynamic web page and its objects’ attributes are loaded from the database to the blocks, they will always be loaded in some repetitive structure within the block. However, attribute-containers are allowed to have empty text or some irrelevant text (the concept of optional

labels, as explained above). The only condition here is that the attribute-container must at least be present in each of the repetitive blocks, even if its empty inside.

The proposed algorithm consists of the following steps:

1) *Constructing an Annotated Tree*: Using the entire DOM tree of the web page, we create a corresponding annotated tree, with each node of the annotated tree, representing an HTML tag in the DOM tree, as shown in Fig. 1. Each node in the annotated tree contains the following fields:

```
{
  "parent" : "The parent of this node",
  "children" : "A list of this node's
    children ids",
  "tagName" : "The corresponding HTML tag
    name",
  "ownLabel" : "This node's own label i.e.
    the predicted label of the
    underlying appended text",
  "descendantsLabelsCount" : "A dictionary
    containing the label count of all
    the descendants",
  "originalPath" : "The path of the
    corresponding HTML tag of this node
    in the original DOM tree"
}
```

While constructing this tree, we determine the field “*ownLabel*” for each node by classifying the text found underneath each node (by appending the text of all the descendants recursively, in a left to right fashion), which is then percolated to the “*descendantsLabelsCount*” of all its ancestors. We only consider the *body* tag in HTML as anything else is usually script or meta data which is irrelevant for us.

2) *Finding hits*: We find all such lowest common ancestral nodes in the annotated tree, which have all the compulsory labels ($\{l_c\}$) present in their subtree, at least once. Starting from the root node, if a node satisfies the condition that it has each of the compulsory labels distributed amongst its descendants, at least once (which is checked by the “*descendantsLabelsCount*” field), we repeat the same procedure on all its children satisfying the same condition, recursively. If no children of the current node satisfy this condition, we have reached a lowest common ancestral node. We call such nodes **hits** and these are the tentative repeating blocks. The *article* shown in Fig. 1 is an example of one such hit as it contains all the three labels (if *TITLE*, *DESC* and *TAGS* are the compulsory labels and the remaining three are optional labels) at least once and it is also a lowest common ancestral node.

3) *Grouping the hits*: A **group** is defined as a collection of nodes having the same *tag_name_path*. The *tag_name_path* is a string defined for every node in the annotated tree, whose value is equal to the concatenation of the tag names of all the nodes between the root node of the DOM tree and itself, in order. For example, the path to reach an *article*, having its *tag_name_path* equal to “*bodydivdivdivsectionarticle*”, from the root node is *body* -> *div* -> *div* -> *div* -> *section* ->

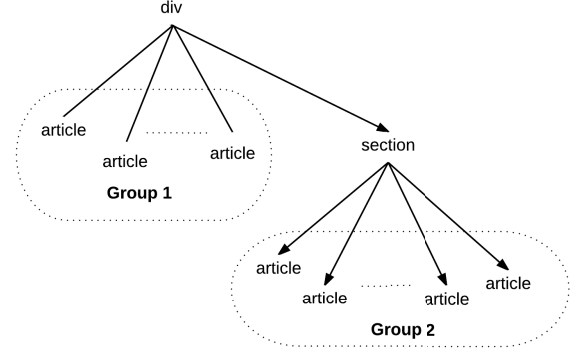


Fig. 2. Grouping the Hits

article. Now, we use hashing based on the tag-name-path, to form multiple groups of hits. An example of grouping is shown in Fig. 2.

4) *Pruning the hits and deleting erroneous hits*: In this step, we extract the repeating substructure from each of the hits in a group, and prune the rest. In other words, any node n_i inside a hit's subtree that does not repeat in every other hit's subtree of the same group, at the corresponding position (which is calculated using this node's internal path or the path from its hit to itself), is pruned along with its descendants. This is done because of the following reasons:

- The unnecessary labels assigned to the nodes of the non-repetitive structure in every hit's subtree are eliminated, which leaves only the relevant labels assigned to the repeating nodes left (which is desired, as the user will always require us to extract attributes, whose attribute-containers are present in all the repetitive blocks).
- It is now possible to find the missed siblings of these hits, that were left out in step number 2, because one of the steps in finding the missed siblings in a group is comparing the repetitive structure of that group with a potential candidate (as explained in the next step).

An example of pruning is shown in the Fig. 3, where every child of the second *div* of each *article* is pruned because each of these children do not repeat (at the corresponding position) in every other member of the same group (the other two *articles*, in this case). To implement this, we calculate the internal paths of the descendants of every hit, put them in a hash table, and keep only the nodes having internal paths with count equal to the total number of members in the group. Moreover, while pruning a node, the “*descendantsLabelsCount*” field of all its ancestors is updated because some of the nodes have now been deleted/pruned and the count of their underlying labels needs to be subtracted from the “*descendantsLabelsCount*” field of each of its ancestors. After this step, the hits of this group which no longer have all the compulsory labels in their subtree (found using “*descendantsLabelsCount*”) are pruned, as such hits were detected because of the wrong classification of some non-repetitive nodes as compulsory labels.

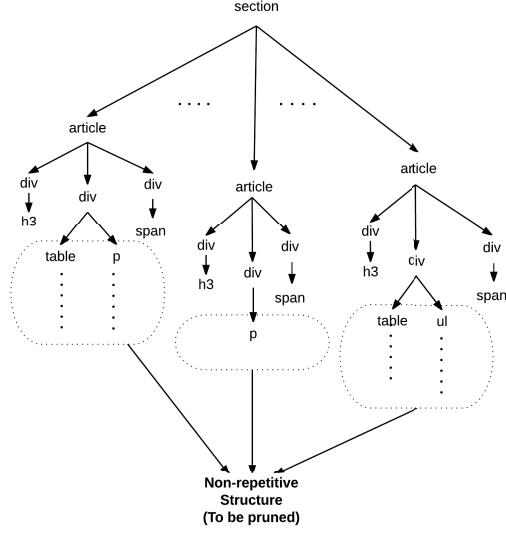


Fig. 3. Pruning the Hits

Another point to note here is that the groups having just one hit are ignored in this step because there is no way to find the repetitive structure in such groups. Also, entire groups can be ignored if all their hits are pruned.

5) *Finding the missed siblings*: This step involves finding the valid missed siblings of the hits of each group, that were missed in step number 2, due to not containing all the compulsory labels in their subtree. This could happen due to two reasons:

- The classifier incorrectly labeled some of their descendants, due to which all the compulsory tags could not be found.
- The values of some of the attributes associated with the compulsory labels were missing in these blocks. For example, if *COUPON_CODE* is made a compulsory label and the value of the offer-coupon-code attribute is null for some of the offers, these offer-blocks will be missed.

The missed siblings are found by the following steps:

- First, the *tag_name_path* is determined for each of the groups (all hits within a group will have the same *tag_name_path*).
- Now, the entire DOM tree is traversed again, in a DFS fashion and for each node n_k , it is checked if the value of its *tag_name_path* matches with the *tag_name_path* of any of the groups.
- If this value matches with that of a particular group, G_j , then it is checked if the **repeating subtree structure** (the common subtree structure of each of the members of a group after pruning) of G_j (which is going to be the same for all the members of G_j after step 4) is a subset of the subtree structure of n_k .
- If it is a subset, then all the non-repeating structure of n_k , as compared to the repeating subtree structure of G_j is

pruned, such that the subtree structure of n_k now matches with the repeating subtree structure of G_j .

e) Finally, n_k is added to G_j .

Fig. 4 describes this process, diagrammatically. The output of this step is the increased (or the same) number of members in each of the groups.

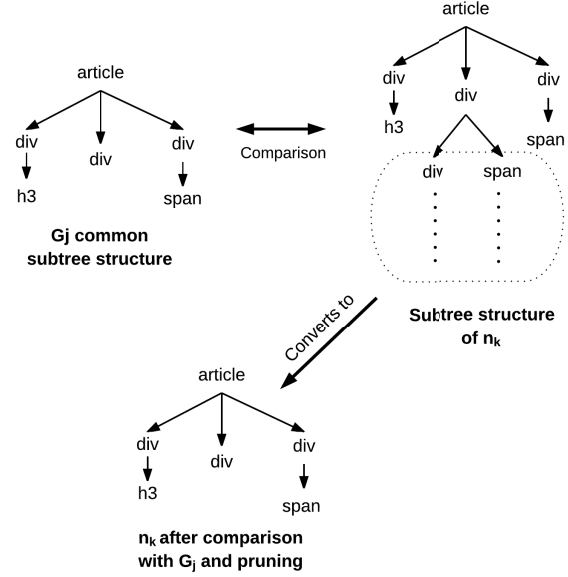


Fig. 4. Finding the missed siblings

6) *Detecting the attributes*: Before proceeding with this step, a key observation is made. Any piece of text that repeats in each of the repetitive blocks of a group (static data) would not be the value of one of the attributes that the user requires. To remove this static data, we extract all the text at the leaf nodes of each of the repetitive blocks in every group, and store (in an array) all such text that lies in more than 95% of the blocks in that group. Some examples of such text are: “Share on Facebook, Twitter, Google+”, or “Report”.

Now, we will find the required attributes from all the members of a group. The label of each node, as predicted by the classifier is read from its “ownLabel” field. First, we must find a way to counter the problem of multiple occurrences of the same label within the subtree of each member of a group (Ideally, there should be just one occurrence; an offer can only have one offer-title, for instance). This issue arises because of the following reasons:

- The classifier wrongly labeled some of the nodes.
- When the classifier predicts the label of a node which inherits all its text from its children, it might end up giving a random label to the node. This is because the classifier is not trained on arbitrary concatenations of text. For example, a node having two children with each having *TITLE* and *DESC* labels, respectively, will most likely be labeled as *DESC*, and thus there will be two *DESC* labels.

To solve this, we first find all the **lowest level occurrences** (occurrences that do not possess a child having the same label as the occurrence itself) of each of the labels in the subtree of each member of the group. Then, for each label, we select the occurrence which is repeating the most among all the members of this group. This is done by comparing the count of the internal paths of each of these lowest level occurrences of a particular label, with other occurrences of the same label in the group. The internal path of the occurrence with the maximum count is output as the relative location of this label (or attribute) in the group, after checking that its underlying text is not present in the static text array, that we formed above. This process is repeated for all the labels, and thus, the internal paths of all the attributes/labels (both compulsory and optional) are extracted, for every group. The repetitive subtree structure (with the output labels) of each group is shown in Fig. 5.

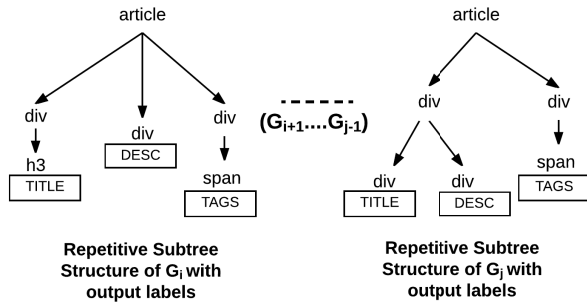


Fig. 5. Detecting the attributes

IV. EXPERIMENTATION AND RESULTS

To test our approach, we experimented with a famous product-offers website in India, called Grabon. It is a well-structured website with a varied collection of offers provided by many well-known banks and digital wallets in India. The offers present in the website have a variety of attributes. We deal with six specific attributes: offer-title, offer-description, offer-coupon-code, offer-expiry, offer-benefits and offer-tags. Since our problem was to deal with the structure changes in a website, we experiment with two versions of various web pages in Grabon, one of July, 2017 and the other one of February, 2016. We observed that the structure of the website in February, 2016 was very different than what it is now. We used Webarchive (<http://web.archive.org/>) to retrieve the old pages.

A. Testing the classifier

The data-set used to train the classifier is extracted from the July, 2017 version of Grabon, using the data collection steps we defined in Section III. The total number of data points after extraction was 2900. After training the classifier (on 80% of the data) and before using it in the proposed algorithm, it was separately tested on 20% of the previously-unseen scraped

data (including that of the label *OTHER*) from Grabon. The accuracy achieved was 99.48%.

The reason behind such a good accuracy in the offers example is that most of the sentences of a particular attribute have similar word lengths and repeating words. For example, most offer-titles have similar words (like “off”, “cashback”, “discount” etc.) and their word-lengths would also be similar to one another (“Get 20% off on KFC” and “Avail 25% cashback on Dominos.”, for instance). Similarly, most offer-descriptions have larger lengths than the offer-titles. These properties are well-used by the classifier to make its predictions and thus, is giving us a very high accuracy.

However, the high accuracy of the classifier is not completely responsible for the high accuracy of our algorithm. This is because the classifier can only label the text of the required attributes and the text belonging to the *OTHER* label (extracted from the leaf nodes), very accurately. We still need to deal with the arbitrary labels assigned by the classifier to the internal nodes, that do not represent any attribute-containers, and thus lead to a repetition of labels, most of which are incorrect. This is taken care by the algorithm and both of them work in conjunction to achieve high overall accuracy.

B. Testing the proposed algorithm

Our problem statement can broadly be divided into two parts: extracting blocks and retrieving their internal attributes. So, it is important to formulate and analyze the results of both these subproblems separately.

We test the proposed algorithm on both the versions (V_1 : July, 2017 and V_2 : Feb, 2016) of five different web pages in Grabon - three digital wallets (Paytm, Mobikwik and Freecharge) and two banks (SBI and ICICI) and tabulate the results in Table I and Table II. The URLs of the web pages used for extraction are given in Table III. The inputs given to the algorithm are the Grabon classifier (trained above), a set of compulsory labels (*MESSAGE*, *DESC*, *BENEFITS*) and a set of optional labels (*TAGS*, *COUPON_CODE*, *EXPIRY*).

Table I displays the results of the first subproblem, i.e. extracting the repetitive blocks from a webpage (offers, in this case). Its column headings TP, FN and FP in Table I correspond to True Positives (the number of valid extracted blocks), False Negatives (the number of missed blocks) and False Positives (the invalid blocks, such as a substructure of a valid block which might contain all the compulsory labels and was mistaken for a valid block), respectively.

Table II contains the results of the second subproblem, i.e. extracting the attributes from the valid blocks found above. It should be noted that since all the members of a group will have the same subtree structure after step 6 of the algorithm, the output internal paths of the attributes contained in them will be the same for all the members of a group. Hence, while defining the accuracy metrics for attribute-extraction, it is more logical to measure the accuracy of the output internal path of an attribute by a group, rather than a single member of that group. (For instance, if a group has 150 members, and the offer-title in the group’s repeating subtree structure

TABLE I
EXTRACTED BLOCK RESULTS

	V ₁ : July, 2017			V ₂ : February, 2016		
Offer-Page	TP	FN	FP	TP	FN	FP
Paytm Coupons	170	0	6	137	0	0
SBI Offers	49	0	0	15	0	0
Freecharge Coupons	38	1	4	50	0	0
Mobikwik Coupons	49	2	10	45	0	0
ICICI Offers	60	0	8	28	0	0
Total	366	3	28	275	0	0
Recall	366/(366+3) = 99.19%			275/(275+0) = 100%		
Precision	366/(366+28) = 92.89%			275/(275+0) = 100%		

is wrongly located, then it will be wrongly located for each of the 150 members of the group and hence, groups are a better representation to measure accuracy than its constituent members). So, the numbers in Table II represent groups, rather than offers, which is why they might appear low in magnitude (a web-page contains 2-3 groups, in general). In the table, an attribute a_i is a True Positive (TP), when it has been assigned a correct internal path inside a block. It is a False Positive (FP), when a wrong internal path was assigned to the attribute and a False Negative (FN), when the correct internal path was not assigned to the attribute (includes the case when it was missed). Lastly, *NA* refers to an attribute which is not present in the web page.

Finally, the recall and the precision are calculated, using their standard formulas,

$$Recall = \frac{TP}{TP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

C. Interpretation of Results

It can be observed from the table that the precision and recall of the first subproblem is very high, especially in version 2, which was not used in the training of the classifier. Mostly, the false positives are detected, when a non-repeating structure (such as an offer-description, which is different in structure for every offer) contains all the compulsory tags, repeats in few of the blocks, and thus are chosen as hits. These false positives in the table can be reduced by increasing the number of compulsory tags, so that more constraints can be imposed on each of the blocks being detected.

TABLE II
EXTRACTED ATTRIBUTE RESULTS

		V ₁ : July, 2017			V ₂ : Feb, 2016		
Offer-page	Attribute	TP	FN	FP	TP	FN	FP
Paytm Coupons	offer-title	2	0	0	1	0	0
	offer-description	2	0	0	1	0	0
	offer-benefits	2	0	0	1	0	0
	offer-coupon-code	1	1	0	0	1	0
	offer-expiry	1	1	1	1	0	0
	offer-tags	2	0	0	NA	NA	NA
SBI Offers	offer-title	2	0	0	1	0	0
	offer-description	2	0	0	1	0	0
	offer-benefits	2	0	0	1	0	0
	offer-coupon-code	2	0	0	0	1	0
	offer-expiry	2	0	0	1	0	0
	offer-tags	2	0	0	NA	NA	NA
Freecharge Coupons	offer-title	3	0	0	1	0	0
	offer-description	3	0	0	1	0	0
	offer-benefits	2	1	0	1	0	0
	offer-coupon-code	2	1	0	0	1	0
	offer-expiry	3	0	0	1	0	0
	offer-tags	2	0	1	NA	NA	NA
Mobikwik Coupons	offer-title	1	1	1	1	0	0
	offer-description	1	1	1	1	0	0
	offer-benefits	2	0	0	1	0	0
	offer-coupon-code	2	0	0	0	1	0
	offer-expiry	2	0	0	1	0	0
	offer-tags	1	1	1	NA	NA	NA
ICICI Offers	offer-title	2	0	0	1	0	0
	offer-description	2	0	0	1	0	0
	offer-benefits	2	0	0	1	0	0
	offer-coupon-code	2	0	0	0	1	0
	offer-expiry	2	0	0	1	0	0
	offer-tags	1	1	1	NA	NA	NA
Total		57	8	6	20	5	0
Recall		57/(57+8) = 87.69%			20/(20+5) = 75%		
Precision		57/(57+6) = 90.4%			20/(20+0) = 100%		

For the second subproblem, we observe that the precision is high, especially in the version 2, but the recall is relatively low. Also, it can be seen that all the attributes, except the offer-coupon-code are successfully extracted from the website in this version (with 100% recall and precision). It was observed on further analysis, that the offer-coupon-code attribute, represented by the attribute-container *small*, was found only in some of the repetitive blocks. This led to the pruning of this node, in step 4 of the algorithm, and thus it was never output in any of valid blocks of the 5 web pages. This violates the condition that the attributes chosen by the user must not

TABLE III
OLD AND NEW VERSIONS OF GRABON: URLS

Offer-page	v1 : July, 2017 (Base URL : http://www.grabon.in)	v2 : February, 2016 (Base URL : https://web.archive.org/web)
Paytm Coupons	/paytm-coupons/	/20160228213137/http://www.grabon.in:80/paytm-coupons/
SBI Offers	/sbi-offers/	/20160220075107/http://www.grabon.in:80/sbi-offers/
Freecharge Coupons	/freecharge -coupons/	/20160226204605/http://www.grabon.in:80/freecharge-coupons/
Mobikwik Coupons	/mobikwik-coupons/	/20160228231809/http://www.grabon.in:80/mobikwik-coupons/
ICICI Offers	/icici-offers/	/20160221084047/http://www.grabon.in/icici-offers/

lie in the non-repeating structure of a block, (as mentioned before in Section III-B). Also, sometimes, two hits (detected in Step 2 of the algorithm) having very different structures might be put together in the same group (Step 3), because we only match the values of their *tag-name-paths* while grouping. This leads to the deletion of some of the attribute-containers while pruning in Step 4, and thus, a failure to detect some attributes. This is another reason for a low recall in Table II.

V. CONCLUSION

In this paper, we tried to solve the information extraction problem for web pages consisting of repetitive blocks. The first part of the problem was extracting these blocks and the second part was retrieving their constituent attributes. The high accuracy (precision and recall) we achieved for an older version of a website, especially for the first subproblem, clearly indicates that our approach is adaptive to the changing structure of a website. Since our algorithm is run infrequently (only when a structure change in the website is detected), our primary focus in this paper was to achieve high accuracy. In the future, we will try to decrease the execution time of our approach by more efficient tree traversals and node searches (currently, it takes around 4 mins to extract offers from a web page of size 1MB, containing around 170 offers). We will also try to deal with some of the limitations and assumptions of our approach, described in the previous sections.

REFERENCES

- [1] N. Kushmerick, D. S. Weld, and R. Doorenbos, "Wrapper induction for information extraction," 1997.
- [2] N. Kushmerick, "Wrapper induction: Efficiency and expressiveness," *Artificial Intelligence*, vol. 118, no. 1-2, pp. 15–68, 2000.

- [3] C.-N. Hsu and M.-T. Dung, "Generating finite-state transducers for semi-structured data extraction from the web," *Information systems*, vol. 23, no. 8, pp. 521–538, 1998.
- [4] I. Muslea, S. Minton, and C. Knoblock, "A hierarchical approach to wrapper induction," in *Proceedings of the third annual conference on Autonomous Agents*, pp. 190–197, ACM, 1999.
- [5] A. L. Gentile, Z. Zhang, I. Augenstein, and F. Ciravegna, "Unsupervised wrapper induction using linked data," in *Proceedings of the seventh international conference on Knowledge capture*, pp. 41–48, ACM, 2013.
- [6] Y. Zhai and B. Liu, "Web data extraction based on partial tree alignment," in *Proceedings of the 14th international conference on World Wide Web*, pp. 76–85, ACM, 2005.
- [7] W. Petprasit and S. Jaiyen, "Web content extraction based on subject detection and node density," in *Knowledge and Smart Technology (KST), 2015 7th International Conference on*, pp. 121–125, IEEE, 2015.
- [8] Y.-C. Chu, C.-C. Hsu, C.-J. Lee, and Y.-T. Tsai, "Automatic data extraction of websites using data path matching and alignment," in *Digital Information Processing and Communications (ICDIPC), 2015 Fifth International Conference on*, pp. 60–64, IEEE, 2015.
- [9] A. Arasu and H. Garcia-Molina, "Extracting structured data from web pages," in *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, pp. 337–348, ACM, 2003.
- [10] H. Song, S. Giri, and F. Ma, "Data extraction and annotation for dynamic web pages," in *e-Technology, e-Commerce and e-Service, 2004. EEE'04, 2004 IEEE International Conference on*, pp. 499–502, IEEE, 2004.
- [11] K. Lerman, L. Getoor, S. Minton, and C. Knoblock, "Using the structure of web sites for automatic segmentation of tables," in *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*, pp. 119–130, ACM, 2004.